

WS63 OHOS Thread 使用方法

本文档说明 WS63 OHOS 下线程（Thread）的原理、实现机制、完整 API 接口及简单示例。

适用平台：九联星闪开发板，基于海思 WS63E 芯片。

文档更新于 2026-03-12，为本人在开发中总结的经验，如有问题请及时联系王老师（wangpeizheng@sztu.edu.cn）。

一、线程原理

1.1 什么是线程

线程是操作系统进行任务调度的基本单位。在嵌入式 RTOS 中，线程（也称任务）是独立运行的执行流，拥有自己的栈空间和优先级，由内核按调度策略在多个线程之间切换 CPU 使用权。

1.2 多任务与调度

- 多任务：**多个线程并发执行，宏观上“同时”运行，微观上由内核在时间片上轮流调度。
- 优先级调度：**高优先级线程优先获得 CPU；同优先级线程通常采用时间片轮转。
- 抢占式调度：**高优先级就绪线程可抢占正在运行的低优先级线程。

1.3 线程状态

状态	说明
osThreadInactive	未初始化或已销毁
osThreadReady	就绪，等待被调度
osThreadRunning	正在运行
osThreadBlocked	阻塞（等待信号量、延时等）
osThreadTerminated	已终止

1.4 线程属性

创建线程时可指定：**名称**、**栈大小**、**优先级**、**栈内存**（可选）等。栈用于保存局部变量和函数调用现场，栈过小可能导致栈溢出。

二、OHOS Thread 实现机制

2.1 分层架构

WS63 OHOS 的线程接口采用分层设计：

应用层：osThreadNew、osThreadGetId 等 CMSIS-RTOS2 API



适配层：kernel/liteos_m/kal/cmsis/cmsis_liteos2.c



内核层：LiteOS-M (LOS_TaskCreate、LOS_TaskDelay 等)

2.2 CMSIS-RTOS2 适配

- **标准接口**：应用通过 CMSIS-RTOS2 (`cmsis_os2.h`) 调用线程 API，便于跨平台移植。
- **适配实现**：`cmsis_liteos2.c` 将 `osThreadNew` 等接口映射到 LiteOS-M 的 `LOS_TaskCreate` 等原生接口。
- **优先级映射**：CMSIS 优先级（如 `osPriorityNormal=24`）会转换为 LiteOS 内部优先级。

2.3 创建流程

`osThreadNew` 内部流程简述：

1. 校验参数（不能在中断中调用、func 非空等）。
2. 解析 `osThreadAttr_t`（栈大小、优先级、名称等）。
3. 调用 `LOS_TaskCreate` 创建 LiteOS 任务。
4. 返回线程 ID（实际为任务控制块指针）。

2.4 相关源码路径

路径	说明
<code>third_party/cmsis/CMSIS/RTOS2/Include/cmsis_os2.h</code>	CMSIS-RTOS2 接口声明
<code>kernel/liteos_m/kal/cmsis/cmsis_liteos2.c</code>	CMSIS 到 LiteOS 的适配实现
<code>kernel/liteos_m/kal/cmsis/BUILD.gn</code>	CMSIS 模块构建配置

三、Thread API 接口说明

3.1 API 总览

API 名称	说明
<code>osThreadNew</code>	创建一个线程并将其加入活跃线程组中
<code>osThreadGetName</code>	返回指定线程的名字
<code>osThreadGetId</code>	返回当前运行线程的线程 ID
<code>osThreadGetState</code>	返回指定线程的状态
<code>osThreadSetPriority</code>	设置指定线程的优先级
<code>osThreadGetPriority</code>	获取指定线程的优先级
<code>osThreadYield</code>	将运行控制转交给下一个处于 READY 状态的线程

API 名称	说明
osThreadSuspend	挂起指定线程的运行
osThreadResume	恢复指定线程的运行
osThreadDetach	分离线程（终止时存储可回收）
osThreadJoin	等待指定线程终止运行
osThreadExit	终止当前线程的运行
osThreadTerminate	终止指定线程的运行
osThreadGetStackSize	获取指定线程的栈空间大小
osThreadGetStackSpace	获取指定线程的未使用栈空间大小
osThreadGetCount	获取活跃线程数
osThreadEnumerate	枚举活跃线程到数组

3.2 osThreadNew

```
osThreadId_t osThreadNew(osThreadFunc_t func, void *argument, const osThreadAttr_t *attr);
```

创建线程并加入调度。**不能在中断服务中调用。**

参数	描述
func	线程函数 <code>void (*)(void *arg)</code>
argument	传递给线程函数的参数指针
attr	线程属性，NULL 时使用默认值

osThreadAttr_t 结构： name、attr_bits、cb_mem、cb_size、stack_mem、stack_size、priority 等。

3.3 osThreadGetName / osThreadGetId

```
const char *osThreadGetName(osThreadId_t thread_id);  
osThreadId_t osThreadGetId(void);
```

获取线程名称或当前线程 ID。

3.4 osThreadGetState / osThreadSetPriority / osThreadGetPriority

```
osThreadState_t osThreadGetState(osThreadId_t thread_id);  
osStatus_t osThreadSetPriority(osThreadId_t thread_id, osPriority_t priority);  
osPriority_t osThreadGetPriority(osThreadId_t thread_id);
```

查询/设置线程状态与优先级。优先级范围：`osPriorityLow`(8)~`osPriorityRealtime7`(55)，数值越大优先级越高。

3.5 `osThreadYield` / `osThreadSuspend` / `osThreadResume`

```
osStatus_t osThreadYield(void);
osStatus_t osThreadSuspend(osThreadId_t thread_id);
osStatus_t osThreadResume(osThreadId_t thread_id);
```

- `osThreadYield`：主动让出 CPU 给同优先级就绪线程。
- `osThreadSuspend`：挂起指定线程。
- `osThreadResume`：恢复被挂起的线程。

3.6 `osThreadDetach` / `osThreadJoin`

```
osStatus_t osThreadDetach(osThreadId_t thread_id);
osStatus_t osThreadJoin(osThreadId_t thread_id);
```

- `osThreadDetach`：设为分离模式，线程终止后资源可自动回收。
- `osThreadJoin`：阻塞等待指定线程结束。

3.7 `osThreadExit` / `osThreadTerminate`

```
void osThreadExit(void); /* 终止当前线程，不返回 */
osStatus_t osThreadTerminate(osThreadId_t thread_id);
```

- `osThreadExit`：当前线程自行退出。
- `osThreadTerminate`：强制终止指定线程。

3.8 `osThreadGetStackSize` / `osThreadGetStackSpace` / `osThreadGetCount` / `osThreadEnumerate`

```
uint32_t osThreadGetStackSize(osThreadId_t thread_id);
uint32_t osThreadGetStackSpace(osThreadId_t thread_id);
uint32_t osThreadGetCount(void);
uint32_t osThreadEnumerate(osThreadId_t *thread_array, uint32_t array_items);
```

获取栈信息、活跃线程数量及线程 ID 列表。

3.9 返回值 `osStatus_t`

值	说明
<code>osOK</code> (0)	成功
<code>osError</code> (-1)	未指定错误
<code>osErrorTimeout</code> (-2)	超时
<code>osErrorResource</code> (-3)	资源不可用

值	说明
osErrorParameter (-4)	参数错误
osErrorNoMemory (-5)	内存不足
osErrorISR (-6)	不能在中断中调用

四、thread 示例项目

4.1 项目创建方式

使用 `build/scripts/make_app.py` 创建 thread 项目：

```
python3 build/scripts/make_app.py -c thread thread_demo
```

4.2 项目结构

```
applications/sample/wifi-iot/app/thread/
├─ thread.c      # 主源码
└─ BUILD.gn      # 构建脚本
```

五、完整源码

5.1 thread.c

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
#include <stdio.h>
#include "ohos_init.h"
#include "cmsis_os2.h"

void thread_entry(void *arg)
{
    (void)arg;
    int count = 0;
```

```

    while (1) {
        count++;
        printf("Thread %d\r\n", count);
        osDelay(500);
    }
}

static void ThreadTestTask(void)
{
    osThreadAttr_t attr = {
        "thread_sub", 0, NULL, 0, NULL, 1024, osPriorityNormal, 0, 0
    };

    if (osThreadNew(thread_entry, NULL, &attr) == NULL) {
        printf("osThreadNew failed.\r\n");
    }
}

APP_FEATURE_INIT(ThreadTestTask);

```

说明：

- `APP_FEATURE_INIT(ThreadTestTask)`：将 `ThreadTestTask` 注册为应用启动时执行的入口。
- `ThreadTestTask` 通过 `osThreadNew` 创建子线程 `thread_sub`。
- 子线程 `thread_entry` 循环打印 "Thread" 计数，每 500ms 输出一次。

5.2 BUILD.gn

```

# Copyright (c) 2026 SZTU Open Source Organization.
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

static_library("thread_demo") {
    sources = [ "thread.c" ]

    include_dirs = [
        "//commonlibrary/utls_lite/include",
        "//kernel/liteos_m/components/cmsis/2.0",
    ]
}

```

六、配置文件说明

配置文件	修改项
applications/sample/wifi-iot/app/BUILD.gn	在 features 中增加 "thread:thread_demo"
device/soc/hisilicon/ws63v100/sdk/build/config/target_config/ws63/config.py	在 ram_component 中增加 "thread_demo"
device/soc/hisilicon/ws63v100/sdk/libs_url/ws63/cmake/ohos.cmake	在 COMPONENT_LIST 中增加 "thread_demo"

使用 `make_app.py` 创建项目时会自动完成上述配置。

七、编译与烧录

7.1 编译

在源码根目录执行：

```
hb set -p nearlink_dk_3863
hb build -f
```

7.2 固件路径

```
out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-liteos-app_all.fwpkg
```

7.3 烧录步骤

1. 连接开发板，安装 CH340 驱动。
2. 打开 BurnTool，选择串口，芯片选择 **WS63**。
3. 选择 `ws63-liteos-app_all.fwpkg`，勾选 `Auto Burn`、`Auto disconnect`。
4. 点击 Connect，按 RST 开始烧录。
5. 烧录完成后按 Reset 重启，串口波特率 **115200** 查看日志。

八、运行结果

```
Thread 1
Thread 2
Thread 3
...
```

九、参考文档

- [03-环境配置和系统编译烧录文档.md](#)：环境配置、编译、烧录与 hb 命令
- [04-创建新项目的方法.md](#)：创建新示例项目的方法